

Computação Paralela

Optimising program performance
on shared memory programming (OpenMP)

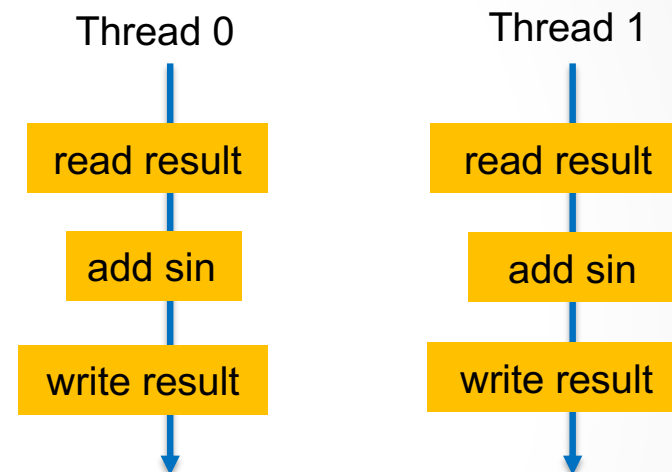
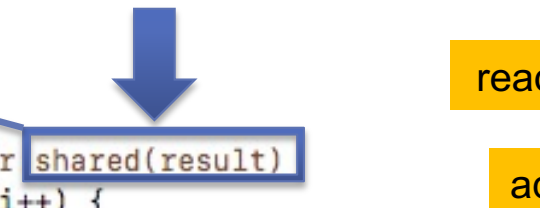
João Luís Sobral
Departamento de Informática
Universidade do Minho

Nov/2025

Recap: Data races (and race conditions)

- A data race **can happen** when two or more threads access (write!) to a **shared memory position**

```
5  int main(){  
6      double result={0};  
7  
8      #pragma omp parallel for shared(result)  
9      for(int i=0; i<1000000;i++) {  
10         result+=sin(i);  
11     }  
12     printf("%f",result);  
13 }
```



- Data races are a sub-set of a broader set of Race Conditions:
 - “a **race condition** is a condition of a program where its behavior depends on relative timing or interleaving”
 - Example:

```
result = f(a);  
g(result);
```

Recap: OpenMP Synchronization

- Solving data races with critical VS atomic

```

5  int main(){
6      double result={0};
7
8      #pragma omp parallel for shared(result)
9      for(int i=0; i<1000000;i++) {
10         result+=sin(i);
11     }
12     printf("%f",result);
13 }

```

Data race

%r12 contains a pointer to a shared memory position

```

.L4: vxorpd %xmm0, %xmm0, %xmm0
     vcvtsi2sdl %ebx, %xmm0, %xmm0 ; convert to double
     call sin
     vmovsd 8(%rsp,%r12), %xmm1 ; load result into %xmm1
     vaddsd %xmm0, %xmm1, %xmm1 ; add
     vmovsd %xmm1, 8(%rsp,%r12) ; update result
     ...
     jne .L4

```

- Critical:** coarse-grain and pessimist

```

call GOMP_critical_start
vmovsd 8(%rsp,%r12), %xmm1 ; load result into %xmm1
vaddsd %xmm0, %xmm1, %xmm1
vmovsd %xmm1, 8(%rsp,%r12) ; update results
call GOMP_critical_end

```

Mutual exclusion: ensures that only one thread executes the critical region (e.g., blocks the calling thread if another thread is still executing the region)

- Atomic:** fine-grain and optimistic (note: simplified code)

```

.L5: vmovsd (%r12), %xmm1
     vaddsd %xmm0, %xmm1, %xmm1
     ...
     lock cmpxchgq %xmm1, (%r12) ; atomic comp & exchange
     cmpq %rax, %rdx
     jne .L5 ; repeat if update failed

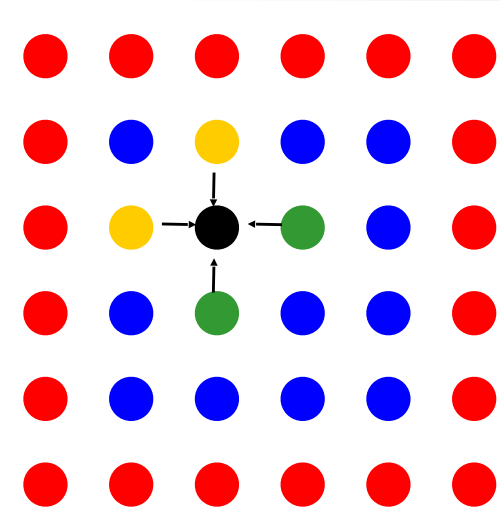
```

Repeat until successful update

Race conditions and data dependencies

Example: stencil computation

```
for(i=1, i<N-1, i++)
  for(j=1, j<N-1, j++)
    A[i,j] = 0,2 x (
      A[i-1,j] +
      A[i,j-1] + A[i,j] + A[i,j+1]
      + A[i+1,j] );
```



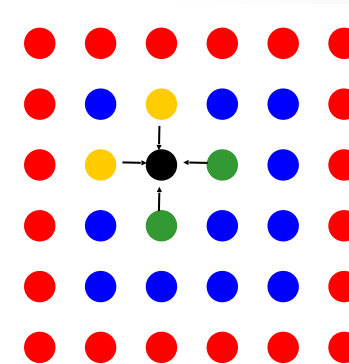
- **Instruction level Parallelism (ILP)**
 - Read multiple values of $A[...]$ from memory in parallel?
 - Perform multiple arithmetic operations in parallel (which?)
 - **Multiply (by 0,2?) and write $A[i,j]$ after all operations are done**
 - **How to improve ILP?**
 - Can we compute $A_{i,j}$ and $A_{i,j+1}$ in parallel?
 - **What dependency constrains the computing of multiple elements of A in parallel?**

```
A[i,j+1] = 0,2 x (
  A[i-1,j+1] +
  A[i,j] + A[i,j+1] + A[i,j+2]
  + A[i+1,j+1] );
```

Race conditions and data dependencies

Example: stencil computation

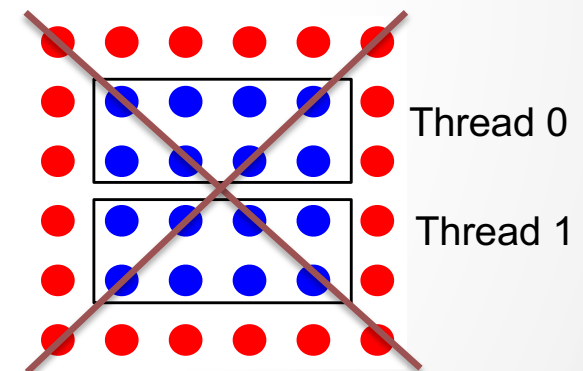
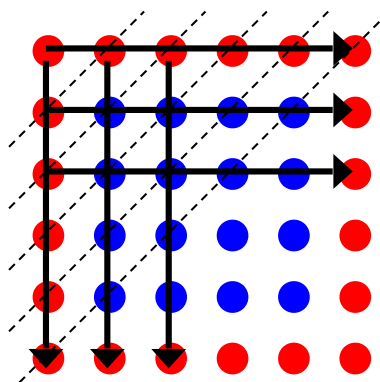
```
for(i=1, i<N-1, i++)  
  for(j=1, j<N-1, j++)  
    A[i,j] = 0,2 x (      A[i-1,j] +  
                        A[i,j-1] + A[i,j] + A[i,j+1]  
                        + A[i+1,j] );
```



- Thread-level [task] parallelism

- Each thread [task] computes a set of rows of the matrix

- Data dependencies?



Does not respect
data dependencies
(introduces a **race
condition**)

Performance of parallel applications

What is the definition of performance?

- Multiple alternatives:
 - Execution time, efficiency, scalability, memory requirement, throughput, latency, project / development costs, portability, reuse potential
 - The relevance of each one depends on the concrete case
 - The most common measure in parallel applications is **execution time**

- **Scalability analysis** of parallel applications:

- **speedup (gain):** execution time of the **best** sequential implementation / execution time of the parallel version

- **Strong scalability** analysis:

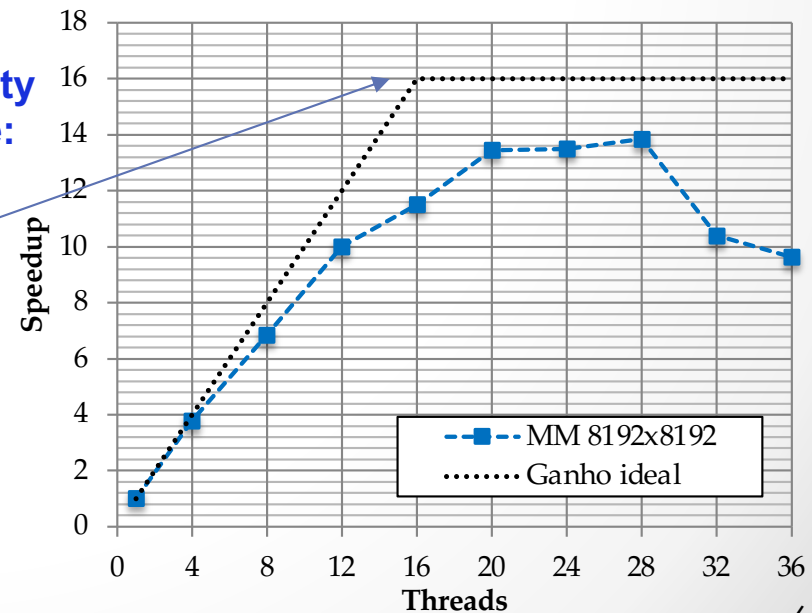
- Speedup increase with PU for a fixed problem data size
 - ideal speedup is proportional to the number of assigned physical PUs

(system with 16 physical PUs in this example)

- **Weak scalability** analysis:

- Increase problem data size as the number of PU increases
 - Ideally the execution time should remain constant

Strong scalability example:



Performance of parallel applications

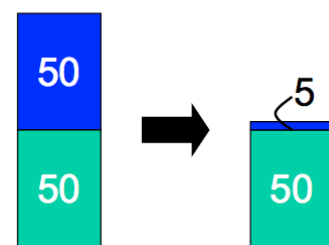
Recap: Amdahl's law (and its impact on the strong scalability)

- The sequential execution time can be divided into:
 - Time doing non-parallelizable work (serial work)
 - Time doing parallelizable work
- The fraction of non-parallelizable work (serial fraction of work) limits the maximum speedup
 - P – number of PU (e.g., #cores)
 - f – serial fraction of work
 - S_p – speedup

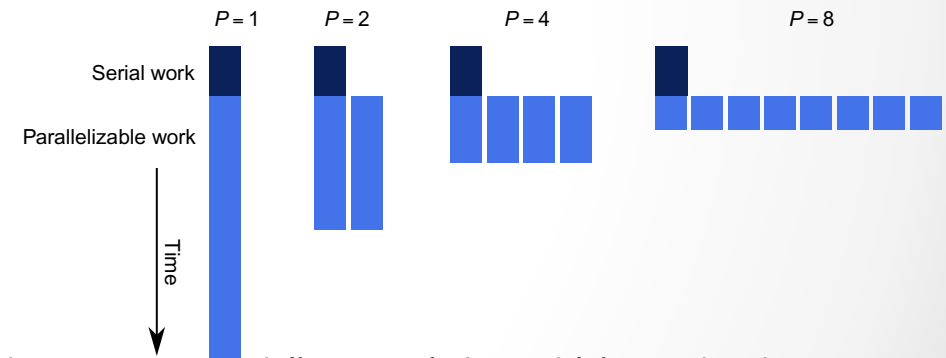
$$S_P \leq \frac{1}{f + (1-f)/P}$$

The maximum speedup is:
1 / serial fraction of work

- Example ($f=0.5$):



10x speedup in
parallelizable work
results in 1.8x
overall speedup



- What fraction of the original computation can be sequential (i.e., serial work) in order to achieve a speedup of 80 with 100 PUs?
 - $80 = 1 / (f + (1-f)/100) \Leftrightarrow f = 0.0025$ (e.g., 0.25%)
- Reinforces the idea that we should prefer algorithms that are suitable for parallel execution:
think parallel!

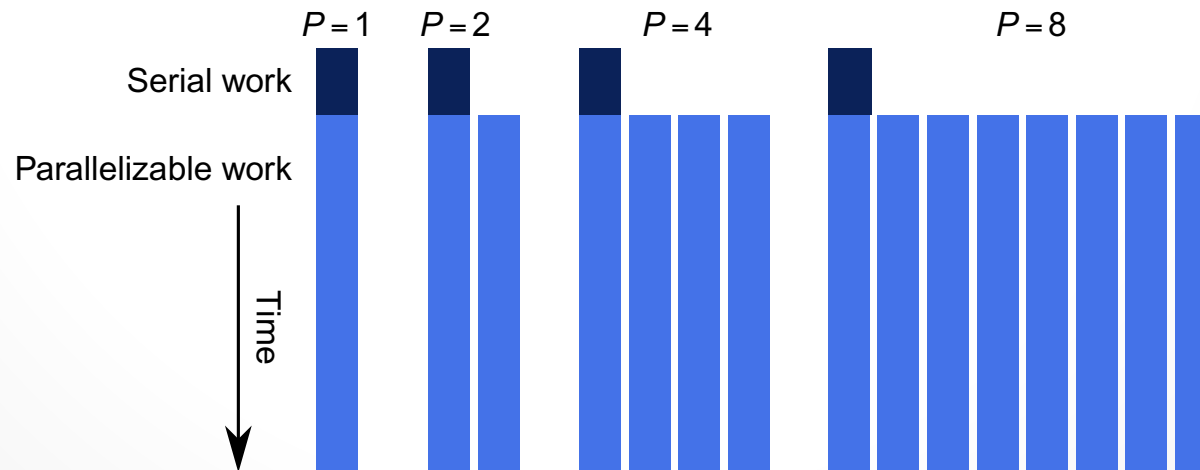
Performance of parallel applications

Speedup anomalies

- Super-linear speedup (superior to the number of PUs):
in most cases it is due to **cache** effects

Gustafson's law (aka weak scalability analysis)

- Increase problem size as the number of PU increases
 - Larger computational resources are usually devoted to larger problem sizes
- The fraction of serial work generally decreases with the problem size
- Weak-scaling example (with ideal speedup)



Generic causes of lack of scalability (in shared memory programming)

Why parallel applications do not have an ideal speedup (1)?

1. Serial work (Amdahl's law)

Computations are preformed serial
non parallelizable work
serialized calls to functions (e.g., rand)

Score-P: Look for **inactive** thread time

2. Parallelism overheads

Additional operations on parallelizable work
thread/task management,
redundant computations, ...

Score-P: Look for **additional** thread time

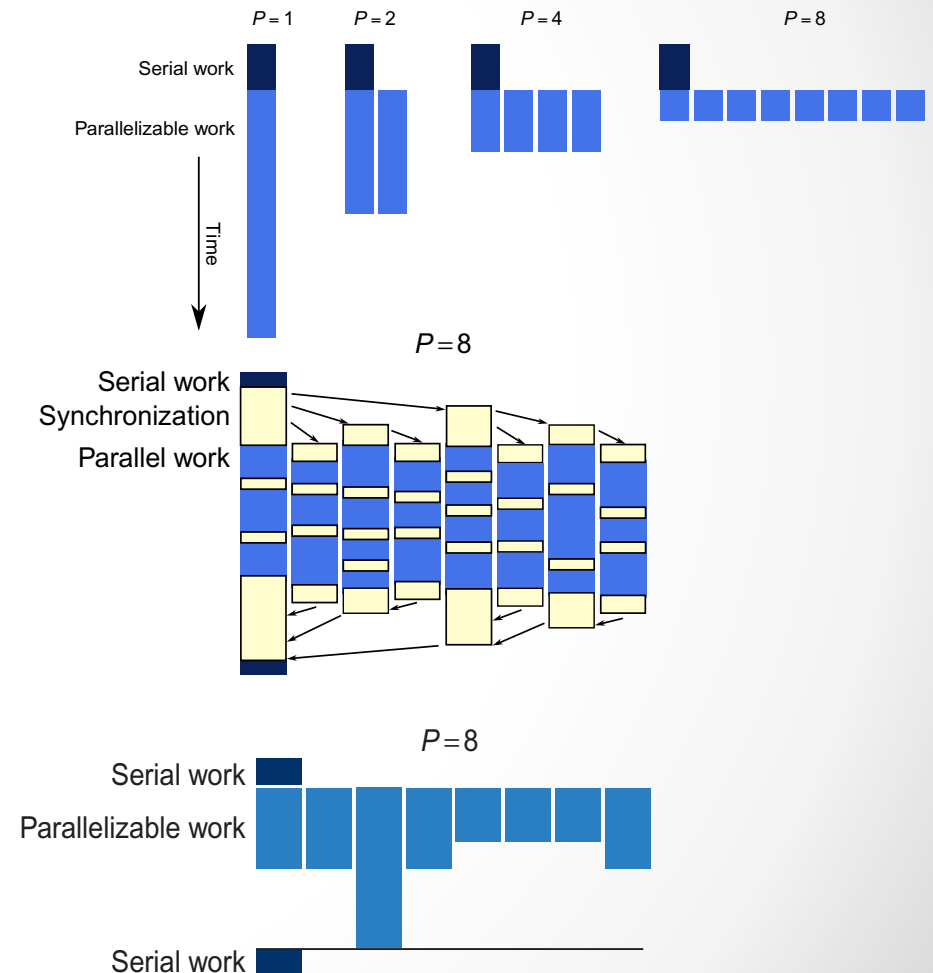
3. Idle time (load imbalance)

Some PUs remain idle while others are still
performing computations

load imbalance,
waiting on a synchronisation point, ...

Score-P: Look for **uneven** thread time

NOTE: execution time is defined by the slowest PU



Parallel Overheads

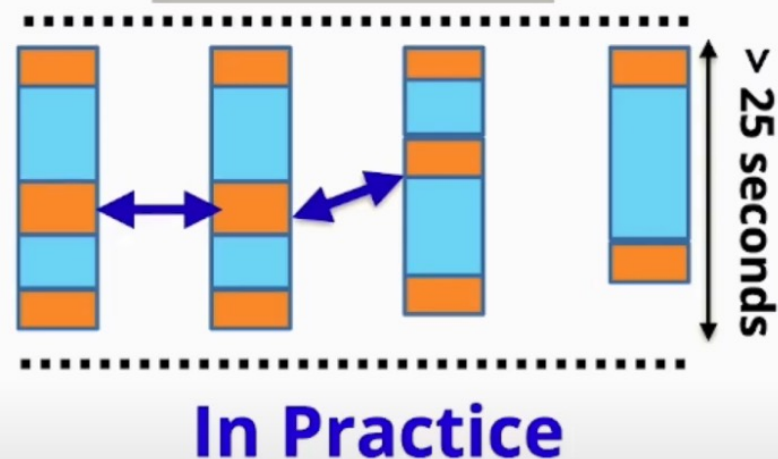
Sequential Program



Parallel Program



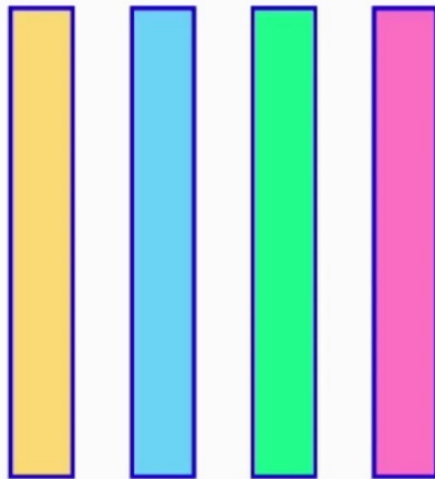
Parallel Program



The goal is to keep the overhead to a minimum



Load Balancing



Ideal situation

- All threads start and finish at the same time
- Shortest execution time



Suboptimal situation

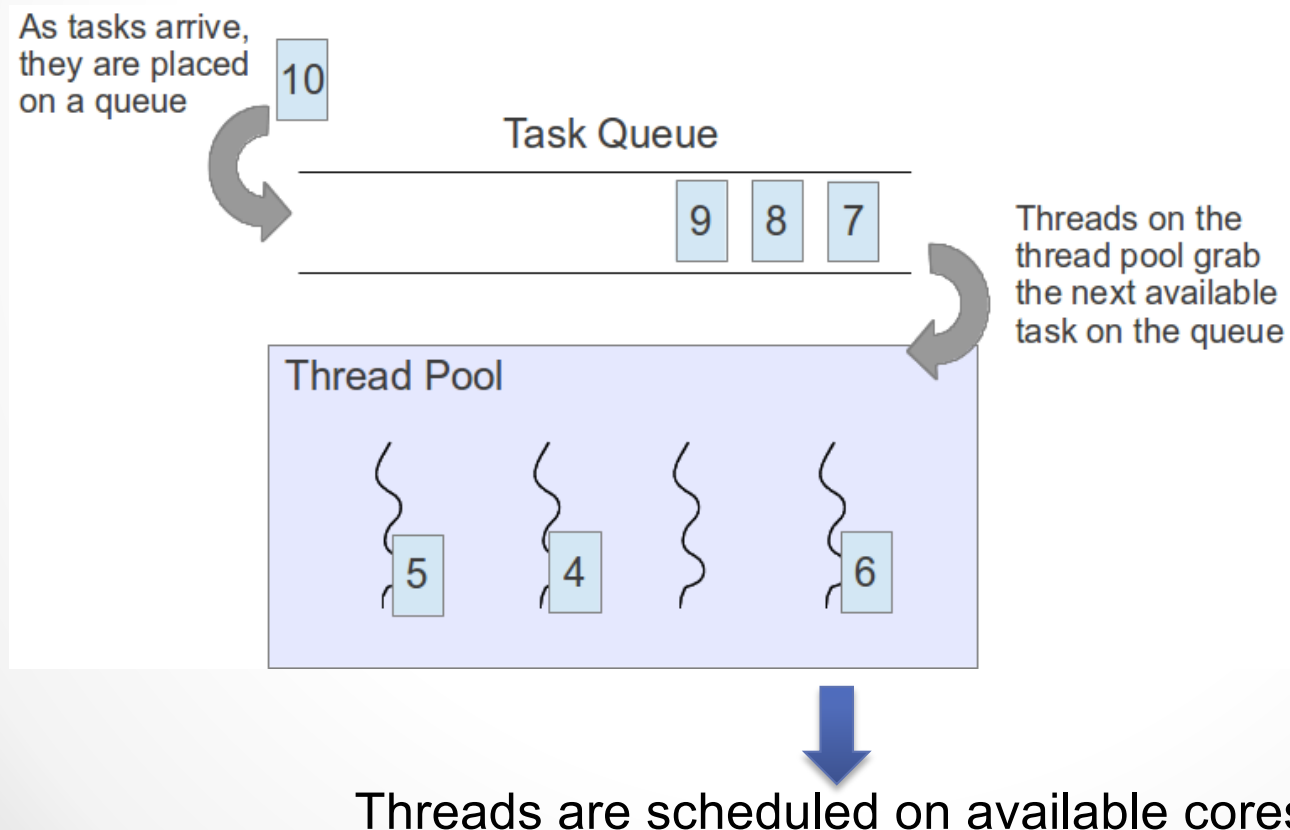
- Threads waste time and energy doing nothing
- Longer execution time

 = thread is waiting ("idle")



Task model

- **Task:** sequence of instructions
- **Thread/process:** execution context for a task
- **PU (processor or core):** hardware that runs a thread/process

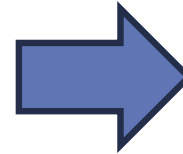


~~Parallel execution of code sections~~ use the task model instead

- ~~Supports heterogeneous tasks:~~ • Implementation with tasks:

```
#pragma omp parallel  
{  
#pragma omp sections  
{  
#pragma omp section  
{  
taskA();  
}  
#pragma omp section  
{  
taskB();  
}  
#pragma omp section  
{  
taskC();  
}  
}
```

Function-parallelism!

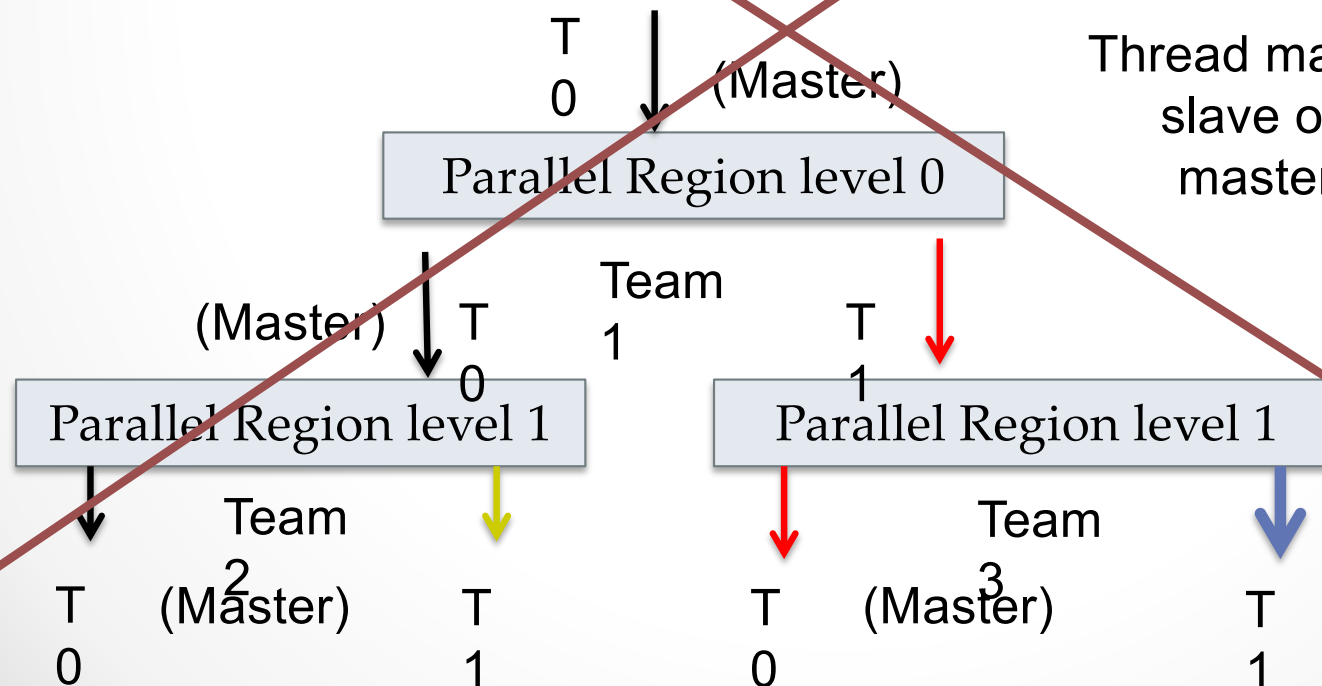


```
#pragma omp parallel  
{  
#pragma omp single  
{  
#pragma omp task  
{  
taskA();  
}  
#pragma omp task  
{  
taskB();  
}  
#pragma omp task  
{  
taskC();  
}  
}  
}
```

~~Nested Parallel Region => (outdated)~~

nested parallelism => use tasks instead

- If a thread in a team executing a parallel region encounters another parallel directive, it creates a new team, and becomes the master of this team;
- If **nested parallelism** is disabled, then no additional team of threads will be created.
- To enable/disable -> `omp_set_nested(x);`
- **Example:**



Thread marked with red is
slave on team 1 but
master on team 2.

Task constructor (from OpenMP 2.5)

```
int fib(int n)
{
    int i, j;
    if (n<2) return n;
    else
    {
        #pragma omp task shared(i) firstprivate(n)
        i=fib(n-1);

        #pragma omp task shared(j) firstprivate(n)
        j=fib(n-2);

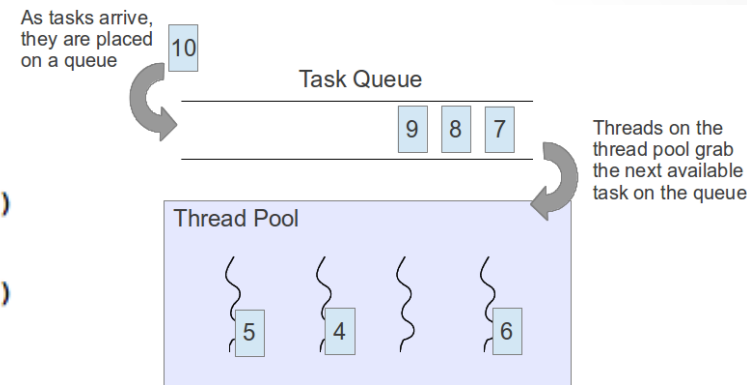
        #pragma omp taskwait
        return i+j;
    }
}

int main()
{
    int n = 10;

    omp_set_num_threads(4);

    #pragma omp parallel shared(n)
    {
        #pragma omp single
        printf ("fib(%d) = %d\n", n, fib(n));
    }
}
```

➤ Execution model:



- The **taskwait** directive ensures that the tasks generated are completed before the return statements.

- Only one thread executes the **single** directive, and hence the call to fib(n),
- **All four threads** will participate in executing the tasks generated.

Execution Tree Exemplified

